

Deployment Guide: React + FastAPI → Windows Server (core.credencerm.com)

Environment: HIPAA-controlled Windows Server — no Python/Node/Git installs allowed.

File Transfer: RDP only (no SFTP, no Git).

Strategy: Bundle everything into a single `.exe` via PyInstaller. IIS handles the domain + SSL.

Prerequisites (on your DEV machine only)

```
# Confirm you have these on your dev machine before starting
python --version      # 3.9+ recommended
node --version        # 16+ recommended
pip install pyinstaller
```

Phase 1 — Prepare FastAPI to Serve React

1.1 — Build the React App

```
# In your React project root
cd frontend/

# Set API base URL to relative — remove all hardcoded localhost:8000 references
# In your .env.production file:
echo "REACT_APP_API_URL=/api" > .env.production

# Build React
npm install
npm run build

# Copy the build output into your FastAPI project's static/ folder
cp -r build/ ../backend/static/
```

⚠ Search your entire frontend codebase for `localhost:8000` and replace with `/api` before building. Any hardcoded URL will silently fail in production.

1.2 — Update FastAPI to Serve the React Build

Add this to your `main.py` (after all `/api/` routes):

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
import os

app = FastAPI()

# --- All your existing API routes go here ---
# @app.get("/api/health")
# def health(): return {"status": "ok"}
# -----

# Serve React static files (must come LAST)
static_dir = os.path.join(os.path.dirname(__file__), "static")
app.mount("/", StaticFiles(directory=static_dir, html=True), name="static")

# Catch-all for React Router (SPA routing)
@app.exception_handler(404)
async def spa_fallback(request, exc):
    return FileResponse(os.path.join(static_dir, "index.html"))
```

1.3 — Handle SQLite Path for Packaged .exe

When PyInstaller bundles your app, the working directory changes. Always resolve paths relative to the `.exe` location:

```
import sys
import os

def get_base_path():
    if getattr(sys, 'frozen', False):
        return os.path.dirname(sys.executable) # running as .exe
    return os.path.dirname(os.path.abspath(__file__)) # running as .py

BASE_DIR = get_base_path()
SQLITE_DB_PATH = os.path.join(BASE_DIR, "data", "app.db")

# Use SQLITE_DB_PATH in your SQLAlchemy or sqlite3 connection
# Example:
```

```
# engine = create_engine(f"sqlite:///{{SQLITE_DB_PATH}}")
```

1.4 — Switch Artiva from MySQL to SQL Server

Your local dev uses MySQL to mock Artiva. Production uses real SQL Server. Handle both via the `.env` file:

```
# database.py
import os
from sqlalchemy import create_engine
from dotenv import load_dotenv

load_dotenv(os.path.join(get_base_path(), ".env"))

ENV = os.getenv("ENV", "development")

if ENV == "production":
    ARTIVA_DB_URL = os.getenv("ARTIVA_DB_URL")      # SQL Server
else:
    ARTIVA_DB_URL = os.getenv("ARTIVA_MYSQL_URL")   # MySQL mock

artiva_engine = create_engine(ARTIVA_DB_URL)
```

MySQL → SQL Server Query Differences

Update any raw SQL queries in your Artiva module:

MySQL	SQL Server
<code>SELECT * FROM `table`</code>	<code>SELECT * FROM [table]</code>
<code>LIMIT 100</code>	<code>SELECT TOP 100 ...</code>
<code>LIMIT 10 OFFSET 20</code>	<code>OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY</code>
<code>AUTO_INCREMENT</code>	<code>IDENTITY(1,1)</code>
<code>NOW()</code>	<code>GETDATE()</code>
<code>IFNULL(x, y)</code>	<code>ISNULL(x, y)</code>
<code>GROUP_CONCAT(x)</code>	<code>STRING_AGG(x, ',')</code>

1.5 — .env File for All DB Credentials

Create `.env` on your dev machine, then copy it to the server via RDP.

This file lives **next to the .exe** and is **never bundled inside it**.

```
# .env
ENV=production

# Artiva Dev DB — SQL Server, standard port
ARTIVA_DB_URL=mssql+pyodbc://username:password@artiva-host/artiva_db?driver=ODBC+

# Artiva Dev DB — SQL Server, custom port (uncomment if needed)
# ARTIVA_DB_URL=mssql+pyodbc://username:password@artiva-host:PORT/artiva_db?drive

# Staging DB — SQL Server
STAGING_DB_URL=mssql+pyodbc://username:password@staging-host/staging_db?driver=OD

# SQLite (resolved automatically via get_base_path)
SQLITE_DB_NAME=app.db

SECRET_KEY=your-secret-key-here
```

Load it in Python:

```
from dotenv import load_dotenv
import os

load_dotenv(os.path.join(get_base_path(), ".env"))

ARTIVA_DB_URL = os.getenv("ARTIVA_DB_URL")
STAGING_DB_URL = os.getenv("STAGING_DB_URL")
```

 **HIPAA:** Add `.env` to `.gitignore`. Never commit credentials.

1.6 — Test DB Connections Before Packaging

Run this on your dev machine with real Artiva credentials before building the `.exe` :

```
# test_connections.py
import pyodbc
import os
from dotenv import load_dotenv
```

```

load_dotenv(".env")

def test(name, conn_str):
    print(f"Testing {name}...")
    try:
        conn = pyodbc.connect(conn_str, timeout=5)
        print(f"   ✓ {name} OK")
        conn.close()
    except Exception as e:
        print(f"   ✗ {name} FAILED: {e}")

test("Artiva", (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    f"SERVER={os.getenv('ARTIVA_HOST')}"
    f"DATABASE={os.getenv('ARTIVA_DB')}"
    f"UID={os.getenv('ARTIVA_USER')}"
    f"PWD={os.getenv('ARTIVA_PASS')}"
))

python test_connections.py

```

Phase 2 — Package with PyInstaller

2.1 — Install Dependencies

```

cd backend/
pip install pyinstaller python-dotenv pyodbc sqlalchemy uvicorn fastapi

```

2.2 — Create the PyInstaller Spec File

```

# Generate base spec
pyinstaller --onefile main.py --name core-app
# Then replace core-app.spec contents with the version below

# core-app.spec
a = Analysis(
    ['main.py'],
    pathex=[],

```

```

    binaries=[],
    datas=[
        ('static', 'static'),    # React build
    ],
    hiddenimports=[
        'uvicorn.logging',
        'uvicorn.loops',
        'uvicorn.loops.auto',
        'uvicorn.protocols',
        'uvicorn.protocols.http',
        'uvicorn.protocols.http.auto',
        'uvicorn.protocols.websockets',
        'uvicorn.protocols.websockets.auto',
        'uvicorn.lifespan',
        'uvicorn.lifespan.on',
        'sqlite3',
        'pyodbc',
        'sqlalchemy.dialects.mssql',
        'sqlalchemy.dialects.mssql.pyodbc',
        'sqlalchemy.dialects.sqlite',
    ],
    noarchive=False,
)

pyz = PYZ(a.pure, a.zipped_data)

exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='core-app',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=True,
    onefile=True,
)

```

2.3 — Build the .exe

```
pyinstaller core-app.spec
```

```
# Output: dist/core-app.exe
```

2.4 — Test Locally Before Copying to Server

```
# Simulate the exact folder layout the server will have
mkdir test-deploy
cp dist/core-app.exe test-deploy/
cp .env test-deploy/
mkdir test-deploy/data
cp data/app.db test-deploy/data/

cd test-deploy/
./core-app.exe --host 127.0.0.1 --port 8000

# Verify:
# http://localhost:8000 → React UI loads
# http://localhost:8000/api/health → { "status": "ok" }
```

Phase 3 — Copy Files to Windows Server via RDP

3.1 — Enable Drive Sharing in RDP

Before connecting, enable local drive sharing so you can copy files across:

1. Open **Remote Desktop Connection** on your dev machine
2. Click **Show Options** → **Local Resources** tab
3. Under **Local devices and resources** click **More...**
4. Check **Drives** (your local C: drive will appear in the server's File Explorer)
5. Connect to the server

Your local drives will appear in the server's File Explorer under:

```
This PC → [your-machine-name] (C:)
```

3.2 — Folder Structure to Create on the Server

In the server's File Explorer or CMD, create this layout:

```
C:\apps\credence-core\  
├─ core-app.exe      ← copy from your dev machine  
├─ .env              ← copy from your dev machine  
├─ nssm.exe          ← download on dev, copy across  
├─ logs\             ← create empty folder  
└─ data\  
    └─ app.db         ← copy your SQLite file
```

:: Run in CMD on the server to create the folders

```
mkdir C:\apps\credence-core
```

```
mkdir C:\apps\credence-core\logs
```

```
mkdir C:\apps\credence-core\data
```

3.3 — Copy Files Across RDP

With drive sharing enabled, drag and drop (or copy/paste) from your local drive into

C:\apps\credence-core\ on the server:

From your dev machine:		To server:
dist\core-app.exe	→	C:\apps\credence-core\core-app.exe
.env	→	C:\apps\credence-core\.env
nssm.exe	→	C:\apps\credence-core\nssm.exe
data\app.db	→	C:\apps\credence-core\data\app.db

 **NSSM download:** Get it from <https://nssm.cc/download> on your dev machine, then copy the `nssm.exe` across via RDP. It's a single `.exe` — no install needed.

3.4 — Verify Files Landed Correctly

In CMD on the server:

```
dir C:\apps\credence-core\  
:: Should show: core-app.exe, .env, nssm.exe  
dir C:\apps\credence-core\data\  
:: Should show: app.db
```

Phase 4 — Run as a Windows Service with NSSM

Open **CMD as Administrator** on the server:

```
cd C:\apps\credence-core\  
  
:: Install service  
nssm.exe install CoreApp "C:\apps\credence-core\core-app.exe"  
  
:: Startup args  
nssm.exe set CoreApp AppParameters "--host 127.0.0.1 --port 8000"  
  
:: Working directory — critical for .env and SQLite to resolve  
nssm.exe set CoreApp AppDirectory "C:\apps\credence-core"  
  
:: Auto-restart on crash  
nssm.exe set CoreApp AppRestartDelay 3000  
  
:: Logging  
nssm.exe set CoreApp AppStdout "C:\apps\credence-core\logs\app.log"  
nssm.exe set CoreApp AppStderr "C:\apps\credence-core\logs\error.log"  
  
:: Start it  
nssm.exe start CoreApp
```

Verify:

```
nssm.exe status CoreApp  
:: Expected: SERVICE_RUNNING
```

Phase 5 — Configure IIS as Reverse Proxy

Ask IT to enable these **Windows Features** (built-in, not third-party):

- Internet Information Services (IIS)
- Application Request Routing (ARR) 3.0
- URL Rewrite Module 2.1

5.1 — Create the IIS Site

Open **PowerShell as Administrator** on the server:

```
Import-Module WebAdministration
```

```
New-Item -ItemType Directory -Path "C:\apps\credence-core\iis-placeholder" -Force
```

```
New-Website -Name "CredenceCore" `
    -PhysicalPath "C:\apps\credence-core\iis-placeholder" `
    -HostHeader "core.credencerm.com" `
    -Port 80
```

5.2 — Create web.config

Create this file at `C:\apps\credence-core\iis-placeholder\web.config` (open Notepad on the server and save it there):

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>

    <proxy enabled="true" />

    <rewrite>
      <rules>
        <rule name="ReverseProxyToApp" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8000/{R:1}" />
          <serverVariables>
            <set name="HTTP_X_FORWARDED_HOST" value="{HTTP_HOST}" />
          </serverVariables>
        </rule>
      </rules>
    </rewrite>

    <httpErrors existingResponse="PassThrough" />

  </system.webServer>
</configuration>
```

5.3 — Enable ARR Proxy (one-time)

```
Set-WebConfigurationProperty `
    -pspath 'MACHINE/WEBROOT/APPHOST' `
```

```
-filter "system.webServer/proxy" `
-name "enabled" `
-value "True"
```

5.4 — Add HTTPS / SSL (Required for HIPAA)

```
# Find your SSL cert in the Windows cert store
$cert = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Subject -like "*c

# Add HTTPS binding
New-WebBinding -Name "CredenceCore" `
               -Protocol "https" `
               -Port 443 `
               -HostHeader "core.credencerm.com" `
               -SslFlags 1

# Assign cert
$binding = Get-WebBinding -Name "CredenceCore" -Protocol "https"
$binding.AddSslCertificate($cert.Thumbprint, "my")
```

If no cert exists yet, ask IT for an internal SSL cert for `core.credencerm.com`.

Phase 6 — Internal DNS

On your internal DNS server, add an **A record**:

```
Name:    core.credencerm.com
Type:    A
Value:    <Windows server internal IP>
TTL:     300
```

Phase 7 — Verify Everything

```
:: Service running?
nssm.exe status CoreApp

:: App responding on port 8000 directly?
curl http://127.0.0.1:8000/api/health
```

```
:: IIS forwarding correctly?  
curl http://core.credencerm.com/api/health
```

From a browser on the internal network:

```
http://core.credencerm.com → React UI loads  
http://core.credencerm.com/api/health → { "status": "ok" }
```

Quick Reference

Task	Command
Build React	<code>npm run build (in frontend/)</code>
Build .exe	<code>pyinstaller core-app.spec (in backend/)</code>
Start service	<code>nssm.exe start CoreApp</code>
Stop service	<code>nssm.exe stop CoreApp</code>
Restart service	<code>nssm.exe restart CoreApp</code>
Check status	<code>nssm.exe status CoreApp</code>
View app logs	<code>type C:\apps\credence-core\logs\app.log</code>
View error logs	<code>type C:\apps\credence-core\logs\error.log</code>
Remove service	<code>nssm.exe remove CoreApp confirm</code>

Re-deploy After Code Changes

```
# --- On your dev machine ---  
cd frontend/  
npm run build  
cp -r build/ ../backend/static/  
  
cd ../backend/  
pyinstaller core-app.spec
```

```
# New exe is at: dist/core-app.exe

# --- RDP into the server ---
# 1. Stop the service first
nssm.exe stop CoreApp

# 2. Copy new core-app.exe from your local drive (via RDP drive share)
#   Overwrite C:\apps\credence-core\core-app.exe

# 3. Restart the service
nssm.exe start CoreApp

# 4. Verify
nssm.exe status CoreApp
curl http://127.0.0.1:8000/api/health
```

Troubleshooting

Problem	Likely Cause	Fix
.exe crashes immediately	Missing dependency or bad path	In CMD: <code>cd C:\apps\credence-core && core-app.exe --host 127.0.0.1 --port 8000</code>
SQLite not found	Path resolving incorrectly	Confirm <code>get_base_path()</code> used everywhere; <code>data\app.db</code> must be next to <code>.exe</code>
Artiva connection fails	ODBC driver missing on server	Ask IT to install ODBC Driver 17 for SQL Server
Artiva wrong syntax	MySQL queries not updated	Replace backticks, <code>LIMIT</code> , <code>NOW()</code> with SQL Server equivalents (see Phase 1.4)
.env not loading	Wrong working directory	Confirm NSSM <code>AppDirectory = C:\apps\credence-core</code>
IIS 502 Bad Gateway	App not running	<code>nssm.exe status CoreApp</code> — restart if stopped
IIS 404 on React routes	Missing catch-all	Confirm <code>web.config</code> rewrite rule exists

React API calls 404	Hardcoded localhost URLs	Search frontend for <code>localhost:8000</code> , replace with <code>/api</code>
Can't copy files via RDP	Drive sharing not enabled	Reconnect RDP with Local Resources → Drives checked

IT Request Checklist

Hand this list to IT — everything they need to action:

1. Enable Windows Feature: Internet Information Services (IIS)
2. Install IIS module: Application Request Routing (ARR) 3.0
<https://www.iis.net/downloads/microsoft/application-request-routing>
3. Install IIS module: URL Rewrite 2.1
<https://www.iis.net/downloads/microsoft/url-rewrite>
4. Install: Microsoft ODBC Driver 17 for SQL Server
<https://learn.microsoft.com/en-us/sql/connect/odbc/download-odbc-driver-for-sq>
5. Provide internal SSL certificate for `core.credencerm.com`
6. Open inbound firewall rules:
 - Port 80 (HTTP)
 - Port 443 (HTTPS)